

Certification Roadmap

What this platform needs at each stage of enterprise adoption.

Level 1 — Presentable to enterprise stakeholders

The minimum to demonstrate a production-ready posture in a technical evaluation. These are gaps that an informed reviewer would flag immediately.

1.1 License file

The repository has no LICENSE file. Without one, the software has no legal terms for use, distribution, or modification. Any enterprise evaluation stops here.

Action: Add MIT or Apache 2.0 license to the repository root.

1.2 Required secrets at startup

The JWT signing secret falls back to a hardcoded default ("change-me-in-production") when the JWT_SECRET environment variable is missing. A production node can start with a known, forgeable secret without any warning.

Action: Make JWT_SECRET a required environment variable. The node must refuse to start if it is missing or matches the default value.

1.3 Key zeroization

Signing keys (Ed25519 and ML-DSA-65) remain in memory after use. If the process crashes or memory is dumped, private key material is recoverable.

The zeroize crate provides a Zeroize trait that overwrites sensitive data on drop. This is a standard requirement for any cryptographic module handling private keys.

Action: Add zeroize dependency. Implement Drop with zeroization on SoftwareSigningProvider and MlDsaSigningProvider key fields.

1.4 Audit trail wired to API

The audit store (`src/audit.rs`) exists with an append-only `AuditStore` trait, `AuditEntry` struct, CSV export, and time-range filtering. However, no API handler actually writes to it. The infrastructure is complete but disconnected.

Action: Add Actix middleware that writes an `AuditEntry` for every API request, using the existing `AuditStore` in `AppState`.

1.5 Fix broken integration test

The PQC migration changed `DagBlock.signature` from `[u8; 64]` to `Vec<u8>`. One integration test file (`tests/store_blocks_api_test.rs`) still uses the old type and fails to compile.

Action: Update the test to use `vec![0u8; 64]` instead of `[0u8; 64]`.

Level 2 — Auditable by a third party

What a security auditor or penetration tester expects before issuing a report. These items don't block a demo but would appear as findings in any formal assessment.

2.1 Property-based testing for cryptography

Unit tests verify specific inputs. Property-based tests (`proptest`, `quickcheck`) verify invariants across random inputs:

- `verify(data, sign(data)) == true` for all data
- `verify(data, sign(other_data)) == false` for all `data != other_data`
- Serialization round-trip: `deserialize(serialize(key)) == key`

This is especially important for the PQC implementation where the algorithm is new and less battle-tested than Ed25519.

Action: Add `proptest` cases for `SoftwareSigningProvider` and `MldsSigningProvider` sign/verify invariants. Add round-trip tests for all serde-annotated structs (`Block`, `Endorsement`, `Transaction`).

2.2 Fuzz testing for parsers

Fuzzing feeds random/malformed input to parsers to find crashes, panics, or undefined behavior. High-value targets:

- Block deserialization from JSON and bincode
- Transaction signature verification with garbage input
- WebSocket message parsing
- P2P message deserialization

Action: Set up cargo-fuzz with targets for `serde_json::from_str::<Block>()`, `serde_json::from_str::<Message>()`, and `Transaction::verify_signature()`.

2.3 Input validation middleware

The API has no global enforcement of:

- Content-Type header validation (reject non-JSON bodies)
- Maximum request body size (prevent memory exhaustion)
- Request rate limiting at the middleware layer (exists as a module but not wired)

Action: Add Actix middleware that enforces Content-Type, max body size (e.g., 1 MB default, configurable), and per-IP rate limiting.

2.4 Production unwrap audit

There are approximately 1,242 `.unwrap()` calls across the source. Most are in test code (acceptable) or use the `unwrap_or_else(|e| e.into_inner())` pattern for poisoned mutexes (acceptable). However, some occur in API handler paths where a panic would crash the request and potentially leak stack traces.

Action: Run `cargo clippy` with the `unwrap_used` lint on `src/api/` and `src/gateway/`. Replace with `map_err()` or `?` propagation. Leave test code as-is.

2.5 Vulnerability disclosure policy

`SECURITY.md` contains a threat model and cryptographic design notes but no process for reporting vulnerabilities. Enterprise customers and auditors expect a documented responsible disclosure process.

Action: Add to `SECURITY.md`: a contact email or form for reporting vulnerabilities, expected response time (e.g., 72 hours), and a statement on coordinated disclosure.

2.6 Coverage enforcement in CI

`cargo-tarpaulin` runs in CI but failures are silent (`continue-on-error: true`). Coverage results are generated but not gated — a PR can merge with 0% coverage.

Action: Set `cargo tarpaulin --fail-under-lines 80` as a required CI check. Keep it as a soft failure only during the initial ramp-up, then make it blocking.

2.7 Encryption at rest documentation

RocksDB stores blocks, transactions, keys, and Raft state on disk. There is no documentation on whether data is encrypted at rest or how to enable it (e.g., RocksDB encryption, filesystem encryption, LUKS).

Action: Document the encryption-at-rest story: whether RocksDB encryption is used, recommended filesystem-level encryption, and key management for disk encryption keys.

2.8 Threat model for consensus

SECURITY.md covers cryptographic threats but not consensus-specific attacks: leader manipulation, block withholding, censorship, eclipse attacks on gossip, or Raft split-brain scenarios.

Action: Extend the threat model with consensus and network-layer threats, mitigations already in place (Raft leader election, gossip signature verification, anti-entropy), and residual risks.

Level 3 — Formal certification

For regulated industries, government contracts, or enterprise customers with compliance departments. Each certification is an independent process with its own auditor, timeline, and cost.

3.1 FIPS 140-3 (Cryptographic Module Validation)

What it certifies: That a specific cryptographic module correctly implements approved algorithms and handles keys securely.

What's needed: - Isolate all cryptographic operations into a defined module boundary - Implement power-up self-tests (known-answer tests for each algorithm) - Key zeroization on all exit paths (normal and error) - Physical security documentation (for hardware modules) or logical security documentation (for software modules) - Submit to a NIST-accredited Cryptographic and Security Testing Laboratory (CSTL)

Current status: The SigningProvider trait provides a clean boundary. Missing: self-tests, formal zeroization, documentation package.

Timeline: 12-18 months from submission. **Cost:** \$50K-150K depending on scope.

Relevance: Required for US government contracts (FISMA). Increasingly referenced by financial regulators in LATAM and EU.

3.2 SOC 2 Type II

What it certifies: That an organization's controls for security, availability, processing integrity, confidentiality, and privacy are designed and operating effectively over a period (typically 6-12 months).

What's needed: - Defined security policies and procedures (access control, incident response, change management) - Audit trail for all system access and changes (partially implemented) - Monitoring and alerting (Prometheus/Grafana infrastructure exists) - Evidence collection over the observation period - Engagement with a licensed CPA firm

Current status: Technical infrastructure (audit logging, monitoring, TLS, ACL) is in place. Missing: organizational policies, observation period, formal auditor engagement.

Timeline: 6-12 months observation period + 2-3 months audit. **Cost:** \$30K-100K.

Relevance: Standard for SaaS and BaaS (Blockchain-as-a-Service) offerings. Expected by enterprise customers in finance and healthcare.

3.3 ISO 27001

What it certifies: That an Information Security Management System (ISMS) meets international standards for risk management, access control, cryptography, operations security, and compliance.

What's needed: - Risk assessment and treatment plan - Information security policy framework (12+ policy documents) - Asset inventory and classification - Internal audit program - Management review process - Engagement with an accredited certification body

Current status: Technical controls (encryption, access control, audit logging) are strong. Missing: policy framework, risk assessment, management processes.

Timeline: 6-12 months preparation + certification audit. **Cost:** \$20K-50K.

Relevance: Recognized globally. Often required for enterprise procurement in EU and LATAM.

3.4 Common Criteria (ISO/IEC 15408)

What it certifies: That a product meets a defined Protection Profile (PP) at a specified Evaluation Assurance Level (EAL 1-7).

What's needed: - Security Target document (ST) defining the product's security claims - Formal functional specification and high-level design - Test documentation and independent testing by the evaluation facility - Vulnerability analysis - For higher EALs: formal methods, source code review

Current status: Not applicable unless targeting government/defense customers requiring EAL4+.

Timeline: 12-24 months. **Cost:** \$100K-500K depending on EAL.

Relevance: Required for some government procurement (US, EU, AU). Overkill for most commercial enterprise use cases.

Summary

Level	Audience	Items	Effort	Blocks deployment?
1		5 items	1 day	Yes

Level	Audience	Items	Effort	Blocks deployment?
	Chamber, enterprise demos			
2	Security auditors, enterprise procurement	8 items	1-2 months	No, but findings expected
3	Regulators, government, compliance	4 certifications	6-24 months	Depends on contract

Recommended path

1. Close Level 1 immediately — these are table stakes
2. Start Level 2 when there is a concrete enterprise customer or audit engagement
3. Pursue Level 3 certifications only when there is a signed contract or regulatory mandate that requires them

Each level builds on the previous one. Nothing in Level 2 contradicts Level 1, and Level 3 assumes Level 2 is complete.